Université Paris Sorbonne Nord
Institut Galilée

Master 1.
Distributed Java Programming
TP3: Threads, Master/Slave patterns

1 File Digest

When a file is downloaded, we usually ask ourselves whether the file was changed (intentionally or accidentally) during transit. For this reason, hashing algorithms are used to produce a 128 or 256-bit number that allows us to *uniquely* [1] identify a file. Take for instance any file in your (Linux) system and do the following:

```
md5sum file
```

This command uses the MD5 algorithm and the result is a 128bit number printed in hexadecimal base[2].
It is also possible to generate a 256bit number with the SHA-256 algorithm:

```
sha256sum file
```

The numbers generated by the hashing algorithms (called *message digest*) are publicly available in the server from where you download your files. Later, you can generate in your own computer the number for the downloaded file. If it does not match, it means that you have downloaded a corrupted/tampered file!
In the file `Digest.java` you will find a class using the `MessageDigest` class from the `security` package in Java that allows you to generate the MD5 and the SHA-256 hash from a file. Test this class and compare the results generated with the Unix commands above.

2 Master-Slave patterns

Computing the MD5/SHA-256 sum from a file may take some time depending on the size of the file. In this part of the TP, you will use threads to compute in parallel (or asynchronously) the hashing of a file. You will experiment with different forms of implementing the master/slave architecture. In this case, the master will be the main program that must use a slave, a thread that computes the digest.

[1] It might be the case that different files produce the same number. This is called a *collision* and they are extremely rare when the algorithms mentioned here are used.
[2] Add or modify a character in your file and repeat the operation. What happens?

1. First, write a class that *implements* the interface `Runnable` or that *extends* the class `Thread`. The method `run` of this class must compute the MD5 or the SHA-256 of a file by calling to the appropriate methods in the class `Digest`.

2. `Test1`. Use the method `join` to print correctly the result of the computation *after* termination. In this case, the master waits until the termination of the slave.

3. `Test2`: *Callback* after the threaded call (**continuation method**). Use the Callback technique in `Example12` of the Course's slides where the slave calls a `static` method (the callback method) defined in the master to inform the result and print it.

4. `Test3`: *Callback* after thread call (**listeners**). In this case, the slave has a field of objects (the listeners), each of them implementing the callback method. As soon as the slave finishes the computation of the digest, it calls the callback method for each listener. For simplicity, we suppose that the only listener is the master. See `Example13.java` in the slides.

5. `Test4`: *Callable*. Use the interfaces `Callable` and `Future` to allow the master to wait for the termination of the slave and print correctly the result.

3 Producers and Consumers

Use the class `BlockingQueue` to implement a queue of strings containing a sequence of file names. Implement a class *Consumer* (see `ProdConQueue.java` in the slides) whose method `run` is an infinite loop that picks an element from the queue, computes the digest and prints the result. Implement also a class *Producer* that populates the queue with several file names. For that, you may use the method `listFiles` of the class `File` to generate a list with the files contained in a given directory.
Your program must create 5 consumers that concurrently will compute the digest for each of the files added by the producer.

3.1 Termination

Most probably the above program will remain running without finishing. Certainly, we can wait for the end of the producer as follows:

```
// Start all the consumers
// Start the producer
Prod.join();
// (*) At this point, the producer has finished
```

In line `(*)` we are sure that the producer has already added all the files to the queue but the consumers are still in an infinite loop. Hence, the program (Java's main thread) cannot finish. We may be tempted to use the method `interrupt` on all the *Consumers* after `Prod.join()`. However, this may cause some problems: there is a chance that the producer is too quick and we will interrupt the consumers before they start processing the queue.
To solve this problem, modify your program as follows:

1. Instead of using the method `take` from `BlockingQueue` use `poll` that waits for a value a given number of time units. Note that this method returns `null` if the specified waiting time elapses before an element is available.

2. Use a shared variable in order to allow all the threads to know whether the producer has already finished. Implement synchronized methods to read/write this variable.

3. Modify the infinite loop in the class `Consumer` to check whether the producer has finished and the queue is empty. In that case, it is safe for the consumer to stop working.

Why do we need to use `poll`? Would it be enough to use `take` and modify the guard of the loop?

4 Streams and concurrency

Finally, generate a stream of file names from the result of `listFiles` of the class `File`. Filter those file names that are not files (i.e., ignore directories). Compute in parallel a map from file names to hash values. Check the elapsed time when a parallel stream is used and compare it with the time needed when the stream is processed sequentially.